

7.29小组模拟面试

1.OpenGL专题

为什么会有OpenGL (OpenGL是干什么的)

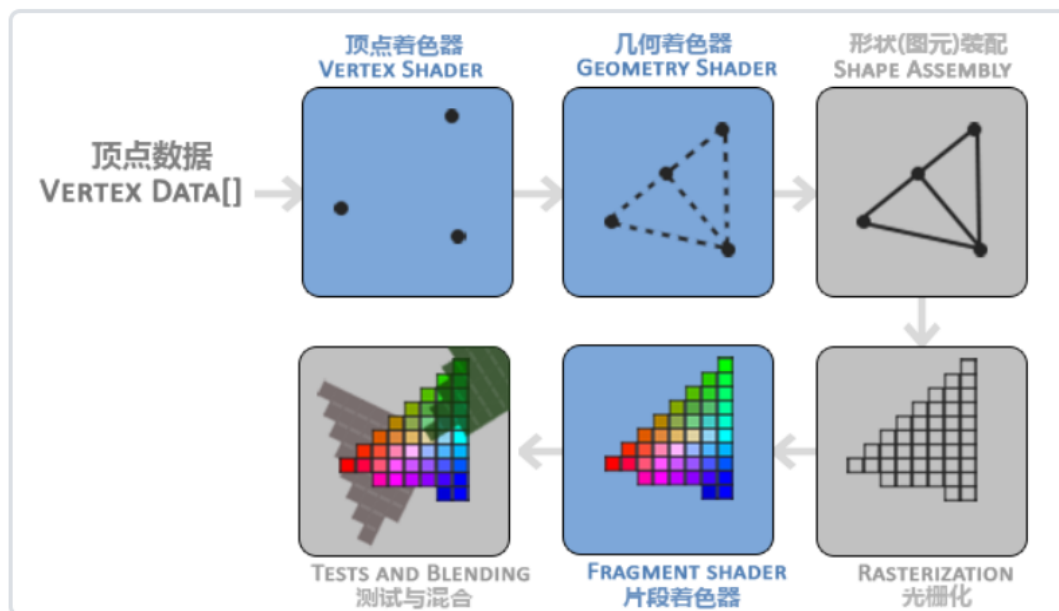
每台计算机都会给程序员提供对外的硬件绘图接口（比如绘制三角形），这些接口不一致，导致程序员代码移植性差，OpenGL就是用来兼容各个版本的接口函数的，**通过判断所在计算机的硬件，采用对应的接口函数绘制，并合理利用GPU进行渲染实现硬件加速**

OpenGL渲染流程（项目代码版）

在项目中，我们自定义了一个类，继承了qt的QOpenGLWidget，并**重写了其三个接口函数（initializeGL, paintGL, resizeGL）**

其中三个函数分别在第一次发生渲染，每次需要渲染，视口大小发生改变时自动调用，具体渲染流程如下

- 首先创建OpenGL上下文，它包含OpenGL状态和资源
- 设置视口，确定渲染区域；
- 把顶点数据（如视频图像的像素点数据）被送入顶点着色器，进行坐标变换等处理（同时获取纹理）；
- 之后经过图元装配，将顶点组装成图元（如三角形）；
- 再进入片段着色器，进行颜色计算和纹理采样等操作；
- 最后进行光栅化，将图元转换为片段（像素），并进行颜色混合和深度测试等操作，最终将处理后的像素绘制到帧缓冲区显示。



你对OpenGL的理解（你是怎么了解到（想到使用OpenGL的）） 动机：

由于我做的项目是一个仿B站视频平台，最开始绘制播放器ui时用的是各种widget容器把控件集成起来，然后再通过调整布局让整体显得美观，当时把整个ui画好了之后我运行程序发现由于我的控件画的比较小，最后视频虽然能播放，但是画质不是很好，所以在网上找了一些方法，当时找到有两个方法，一个说用QPixmap代替QImage直接缩放图片比例显示，因为QPixmap可以吃到GPU硬件加速，它内部采用QPainter进行重绘渲染，使用起来也比较简单，画质确实提高了，但这个方法能做的事比较少，好像**只能进行基础的图像缩放和旋转**。另一个说法就是用专业的API工具OpenGL，它相较于QPixmap渲染2D图片，可以渲染3D模型或物体，通过定义一个视图和摄像头视角，把3D模型的切面显示在视图上，再通过纹理控制和Shader着色器实现精细化渲染，还有很多的库实现一些别的功能（比如自定义顶点着色器实现视频的镜像播放），同时最重要的是OpenGL库是可移植性较强的，Qt也提供了其接口ui（QOpenGLWidget和QOpenGLWindow）以及内嵌api，通过对OpenGL的大致学习，我把原来的显示widget升级为QOpenGLWidget，重写了里面的三个接口函数（initializeGL, paintGL, resizeGL），每次进行拉伸我的视频播放器或者有新的图片到来时，就会由OpenGL先进行渲染并显示，由于也采用GPU硬件加速，所以渲染的结果也是很好的

在OpenGL中实现了自定义Shader吗？

首先明确Shader是什么：着色器

正常情况下我们使用OpenGL是处理3D图像，通过确定摄像机视角初始化视口，获取切面映射，将其抽象为顶点和纹理，在处理过程中采用着色器（可以是自己实现的，也可以用默认的），其中**可编程的几个着色器**如下

顶点着色器：把三维坐标转换到二维视口坐标，通过编程，我们可以转换投影的节点视角，比如对顶点做旋转或别的动态效果（视频镜像播放）

几何着色器：输入顶点组，形成完整图元（点线三角形），抛弃多余图元

片段着色器：负责计算每个像素（片段）的颜色，最终呈现在屏幕上（是滤镜，光照，纹理混合的结果）

在我们的项目中，只是单纯重写了三个qt的QOpenGLWidget的函数，并没有手动实现自定义Shader

其中从顶点着色器----->几何着色器，不仅传递顶点坐标，还传递纹理坐标（如何理解纹理：本质是GPU中存储的一张图像，通过定义纹理坐标（需要加纹理的视口部分），到了片段着色器时，会从图像中取出纹理坐标范围内的纹理图像，把其颜色着色到对应的视口位置），使用纹理可以减少顶点绘制，提高渲染效率

为什么图像渲染不用OpenCV而是用OpenGL

- OpenGL专注于图形渲染，在图形处理和加速方面有优势，能直接与显卡交互，实现高效的硬件加速渲染，适合实时视频播放场景。
- OpenCV侧重于计算机视觉算法，在图形渲染性能上不如OpenGL。

GPU渲染和CPU渲染的比较（opengl渲染和直接渲染的优势，实际就是比较GPU和CPU）

GPU内部有数千个以较低时钟运行的内核，可以执行并发任务，比如我要渲染一张图片，我可以把这个图片的9*9的像素合成一块，交给GPU的某一个内核单元处理，同时由于GPU有数千个内核并行，所以GPU渲染效率比较高

相较之下CPU内核少，但是时钟长，且可以随机访问RAM存储器，因此可以相对轻松渲染出有大量数据的情景，渲染质量比较高（但没GPU快）

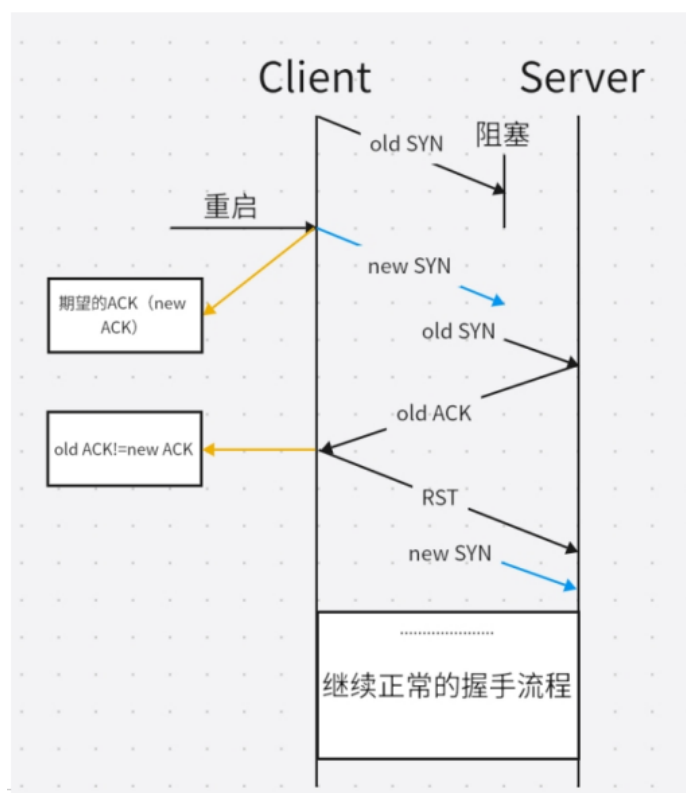
2.什么是万能引用

```
template void func(T&& tmpvalue) {...}
```

本质是个模版，参数是&&，既可以传左值又能传右值，万能引用经常和forward结合，实现完美转发（保留原始值类别：比如我传 &，此时在&&作用下还是&，右值引用一样是&&，左值或右值直接变成&&，都延长了生命周期，且没有发生额外的拷贝）

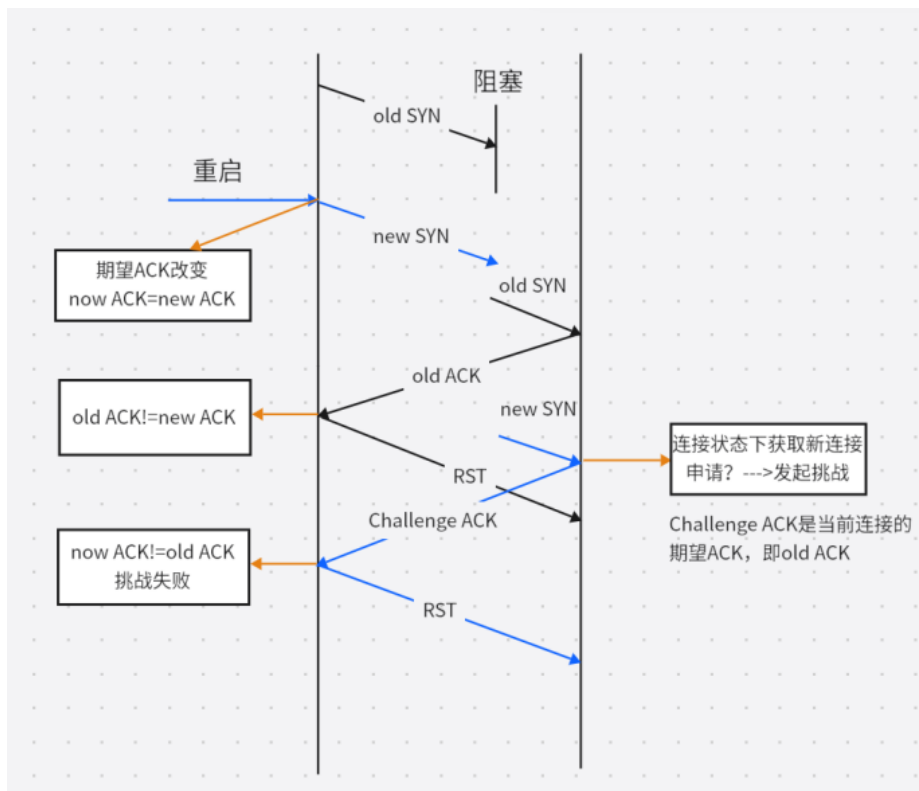
为什么要完美转发：避免不必要的拷贝或移动，提升性能（尤其对于大型对象或资源句柄），且支持把参数原封不动传给其他函数

3.三次握手时，第一次客户端发SYN，同时断开连接（SYN没到服务器），客户端重新登陆并再发SYN，会发生什么？



此时服务器先收到old SYN，回复ACK+SYN，客户端一看不是自己想要的新ACK+SYN，就回RST，此时在发送的报文有两个，RST和new SYN。

- new SYN先到，由于服务器之前已经建立了连接，所以此时会回复刚刚旧连接回的ACK（challenge ACK），看看是不是客户端发错了，因为连接还没断开，此时客户端一看，不是自己想要的，就会再发一个RST断开新连接
- RST先到，此时先断开旧连接，正常进行三次握手

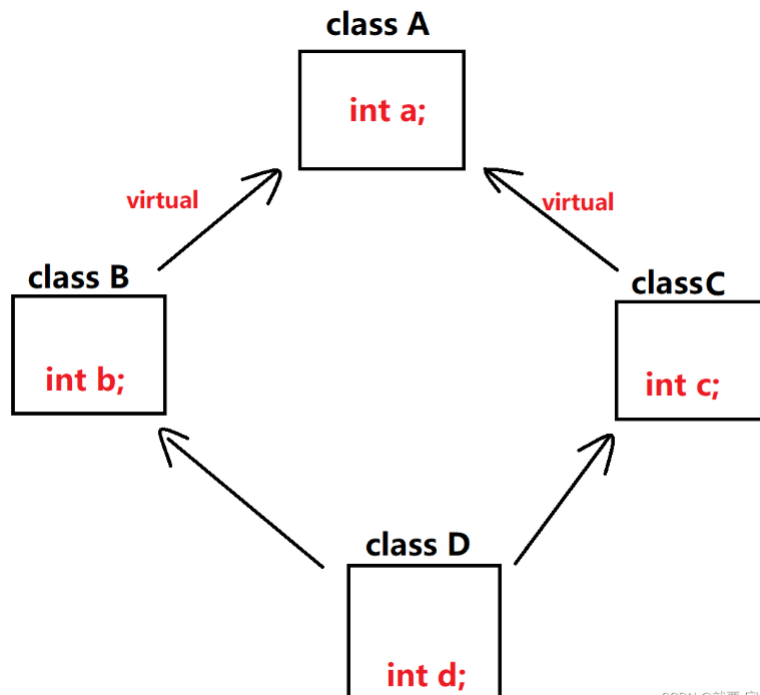


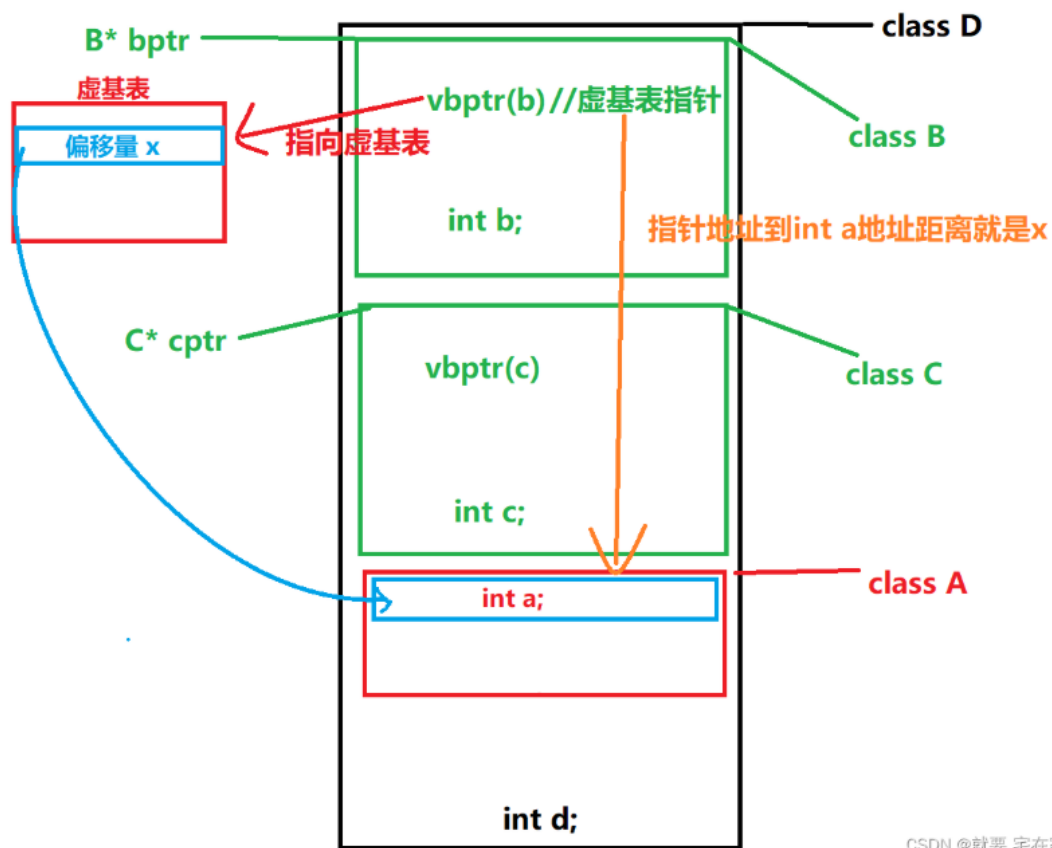
4.什么是多继承？ 如果多继承发生二义性如何解决？

多继承：一个类继承多个类

多继承二义性：一个类继承的两个类有公共爷爷类，此时爷爷类空间被孙子类中存在两份，使用时产生二义性

如何解决（虚继承）：在**父亲层继承爷爷时**加上virtual关键字，表示虚继承





CSDN @就要 宅在家

虚继承如何解决二义性问题：如图虚继承之后，孙子类D多继承两个父亲，此时根据继承先后顺序，在自己的类内出现两个虚基表指针（父亲类继承爷爷类时，所有继承的东西都会放到虚基表中），指针指向虚基表，表中存放两个数据，这两个数据标志了偏移量，通过偏移量能找到爷爷类继承部分的变量或函数在孙子类中位置，由于BC类是同一个爷爷，所以此时两个虚基表中偏移量会指向同一份空间，从而避免了二义性